

INFO8006 Introduction to Artificial Intelligence

Exercise session 2: Games and adversarial search

RECAP

Games

A *game* is defined by

- the *states* $s \in \mathcal{S}$ and the *initial state* $s_1 \in \mathcal{S}$;
- the *actions* $\text{actions}(s) \subseteq \mathcal{A}$ and the *transition model* $s' = \text{result}(s, a)$;
- a function $\text{player}(s) \in \{1, \dots, N\}$ that says whose turn it is in state s ;
- the *terminal states* $T \subseteq \mathcal{S}$, tested by $\text{terminal-test}(s)$, *i.e.* $s \in T$;
- a *utility* $\text{utility}(s, p)$, the payoff of player p when the game ends in $s \in T$.

The *game tree* is defined by s_1 , actions and result. A solution is a *strategy* $\pi_p : \mathcal{S} \rightarrow \mathcal{A}$, a move for every state the opponent can lead to.

Zero-sum games

In a *zero-sum* game, the total payoff is constant. Below, the game is deterministic, turn-taking, two-player, zero-sum and of perfect information: MAX maximizes the utility, MIN minimizes it, and MAX moves first.

Minimax

The *minimax value* of s is the best payoff MAX can secure against an optimal MIN,

$$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a \text{minimax}(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a \text{minimax}(\text{result}(s, a)) & \text{if MIN plays,} \end{cases}$$

over $a \in \text{actions}(s)$, and the optimal move is $\pi^*(s) = \arg \max_a \text{minimax}(\text{result}(s, a))$. Time $O(b^m)$, space $O(bm)$.

α - β pruning

α and β are the best values found so far for MAX and MIN on the path from the root: a MIN node stops as soon as its value $v \leq \alpha$, a MAX node as soon as $v \geq \beta$. The minimax values are unchanged and, with a perfect move ordering, $O(b^{m/2})$ nodes are examined.

H-Minimax

Under time constraints, an *evaluation function* $\text{eval}(s)$ replaces the utility and a *cutoff test* at depth d replaces the terminal test,

$$\text{h-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if cutoff} \\ \max_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MAX plays} \\ \min_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MIN plays.} \end{cases}$$

The evaluation should order states as their minimax values do, and the cutoff apply to *quiescent* states only; what lies beyond the depth limit is hidden (*horizon effect*).

Expectiminimax

Chance nodes map a state-action pair to its outcomes r , of probabilities $P(r)$. With $v(s) = \text{expectiminimax}(s)$,

$$v(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a v(\text{result}(s, a)) & \text{if MAX plays} \\ \min_a v(\text{result}(s, a)) & \text{if MIN plays} \\ \sum_r P(r) v(\text{result}(s, r)) & \text{if CHANCE plays.} \end{cases}$$

Monte Carlo tree search

Each node n keeps the wins $Q(n, p)$ of player p over the random playouts through n and its visits $N(n)$. A round does *selection*, *expansion*, *simulation* and *backpropagation*; selection picks the child n' that maximizes

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}}.$$

In session: exercises 1, 2.

Exercise 1 Tic-tac-toe (AIMA ch. 5)

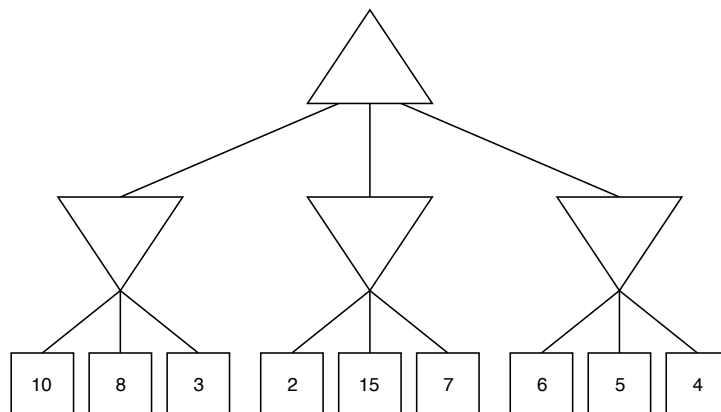
Tic-Tac-Toe is a game for two players, X and O, who take turns marking the cells of a 3×3 grid. The player who succeeds in placing three of their marks in a straight line (horizontal, vertical or diagonal) wins the game. If neither of the players wins before the grid is full, it is a draw.

We consider X as the max player and O as the min player. We define X_n as the number of rows, columns or diagonals with exactly n X's and no O's. Similarly, O_n is the number of rows, columns, or diagonals with just n O's. A position s is terminal if $X_3(s) \geq 1$, $O_3(s) \geq 1$ or if the grid is full. The utility function assigns +1, -1 or 0 to such position, respectively. For non-terminal positions, we use an evaluation function defined as $\text{eval}(s) = 3X_2(s) + X_1(s) - 3O_2(s) - O_1(s)$.

- (a) Define the search problem associated with the Tic-Tac-Toe game.
- (b) Approximately how many possible game states of Tic-Tac-Toe are there?
- (c) Show the whole game tree starting from an empty grid down to depth 2 (one X and one O on the board), taking symmetry into account.
- (d) Annotate your tree with the evaluations of all the positions at depth 2.
- (e) Using the H-Minimax algorithm, annotate your tree with the backed-up values for the positions at depths 1 and 0, and use those values to choose the optimal starting move.
- (f) Is this evaluation function a good heuristic? If not, provide one or more states s for which $\text{eval}(s)$ is misleading.

Exercise 2 Minimax (CS188, Fall 2019)

Consider the zero-sum game tree below. Triangles that point up, such as at the top node (root), represent choices for the maximizing player; triangles that point down represent choices for the minimizing player.



- (a) Assuming both players act optimally, fill in the Minimax value of each node.
- (b) Which nodes can be pruned from the game tree above through α - β pruning? If no nodes can be pruned, explain why not. Assume the search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.
- (c) Again, consider the same zero-sum game tree, except that now, instead of a minimizing player, we have a chance node that will select one of the three values uniformly at random. Fill in the expectiminimax value of each node.

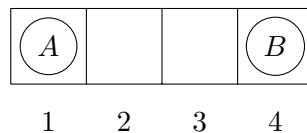
Exercise 3 21 misery game (Exam, January 2019)

The game “21” is played with any number of players who take turns increasing a counter. The counter starts at 1 and each player in turn increases the counter by 1, 2, or 3, but may not exceed 21; the player who says “21” or larger loses.

- Define the search problem associated with the 2-player version of the “21” game.
- For the following, consider the game of “5” (still in its 2-player version), which has the same rules as “21” except that you should not say 5 or more. Show the whole game tree.
- Using the Minimax algorithm, annotate your tree with the backed-up values, and use those values to choose the optimal starting move.

Exercise 4 Leapfrog (AIMA ch. 5)

Consider the following two-player turn-taking game whose initial configuration is shown in the figure below. Player *A* moves first. Each player must move their token to an adjacent free cell in either direction. If the opponent occupies an adjacent cell, then a player may jump over the opponent to the next free cell, if any. For example, if *A* is on 3 and *B* is on 2, then *A* may move back to 1. The game ends when a player reaches the opposite end of the board. If player *A* reaches cell 4 first, then the value of the game to *A* is +1; if player *B* reaches cell 1 first, then the value of the game to *A* is −1.



- Define the search problem associated with this game.
- Draw the complete game tree, using the following conventions:
 - Put each terminal state in a square box and annotate it with its game value.
 - Put loop states (states that already appear on the path to the root) in double square boxes. Since their value is unclear, annotate them with a “?” symbol.
- Explain why the standard minimax algorithm would fail on this game.
- Annotate each node with its backed-up minimax value. Explain how you handled the “?” values and why.
- This 4-cell game can be generalized to n cells for any $n > 2$. Prove that *A* wins if n is even and loses if n is odd.

Exercise 5 Chess and transposition table (AIMA ch. 5)

Suppose you have a chess program that can evaluate 16 million nodes per second.

- Decide on a compact representation of a game state for storage in a transposition table.
- About how many entries can you fit in a 4 GB in-memory table?
- Will that be enough for the three minutes of search allocated for one move?
- How many table lookups can you do in the time it would take to do one evaluation? Suppose that you have a 3.2 GHz machine and that it takes 20 operations to do one lookup on the transposition table.

QUIZ

1. In Monte Carlo Tree Search (MCTS), in the formula

$$\frac{Q(n', p)}{N(n')} + c \sqrt{\frac{\log N(n)}{N(n')}},$$

which of the following is true?

- ☐ The first term encourages the exploitation of higher-reward nodes, while the second encourages the exploration of less-visited nodes.
- ☐ The first term encourages the exploration of less-visited nodes, while the second term encourages the exploitation of higher-reward nodes.
- ☐ The first term encourages the exploitation of highly-visited nodes, while the second term encourages the exploration of lesser-rewarding nodes.
- ☐ The first term encourages the exploration of lesser-rewarding nodes, while the second term encourages the exploitation of highly-visited nodes.

2. In adversarial search,

- ☐ The horizon effect arises when the search is stuck in a cycle.
- ☐ The horizon effect arises when the evaluation function is perfect.
- ☐ The deeper in the tree the evaluation function is buried, the more the quality of the evaluation matters.
- ☐ If not looked deep enough, bad moves may appear as good moves, because their consequences are hidden beyond the search horizon.

3. A quiescent state is

- ☐ A state in which the game will loop indefinitely.
- ☐ A state in which an agent is stuck.
- ☐ A state in which the outcome of a game is unlikely to vary a lot in the near future.
- ☐ A state in which the MAX agent is certain to win.

4. Minimax algorithm

- ☐ Cannot fail more against a sub-optimal agent than an optimal one.
- ☐ May be outperformed by a strategy that exploits a sub-optimal but predictable opponent.
- ☐ Does not ensure that the MAX agent always wins.
- ☐ All of the above.